

EN3160: Image Processing and Computer Vision

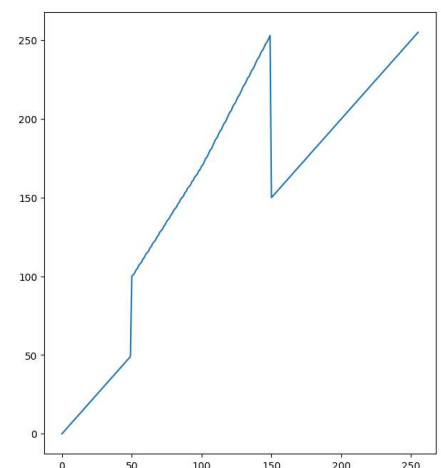
Assignment 01: Intensity Transformations and Neighborhood Filtering

- **Name:** Ayanaja N.B.G.M
- **Index Number:** 230065H
- **GitHub Repository:** https://github.com/ManiiAya/EN3160_Assignment_01

Question 1: Piecewise Linear Intensity Transformations

```
In [114]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve2d
```

```
In [115]: x = [0,50,50,100,150,150,255]
y = [0,50,100,170,255,150,255]
f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)
X = np.arange(256, dtype='uint8')
F = np.interp(X,x,y).astype(np.uint8)
fig, axes = plt.subplots(1,3, figsize=(24,8))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[1].imshow(F[f], cmap='gray', vmin=0, vmax=255)
axes[2].plot(X,F)
plt.show()
```



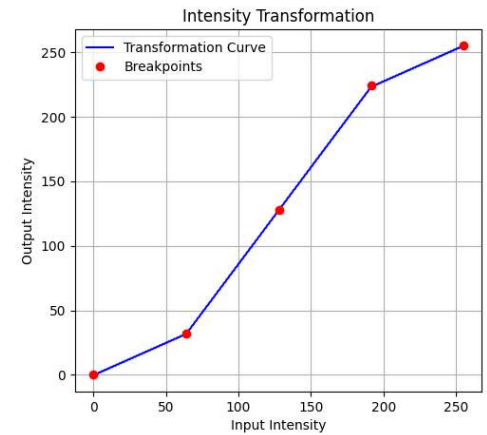
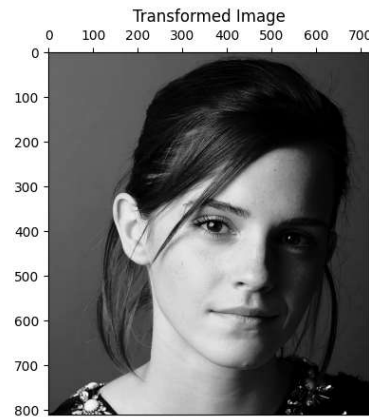
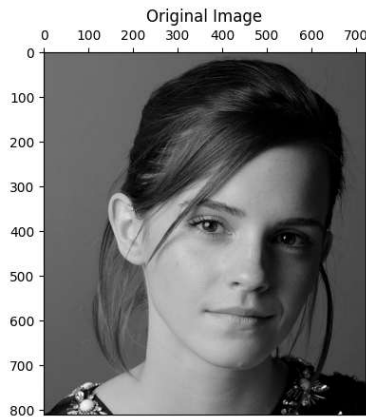
```
In [116]: def intensity_transform(im, breakpoints):
x, y = zip(*breakpoints)
X = np.arange(256, dtype='uint8')
F = np.interp(X, x, y).astype(np.uint8)
return F[im], X, F
```

```
In [117]: f = cv.imread("images/emma.jpg", cv.IMREAD_GRAYSCALE)
breakpoints = [(0, 0), (64, 32), (128, 128), (192, 224), (255, 255)]
transformed_img, X, F = intensity_transform(f, breakpoints)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image");axes[0].xaxis.set_ticks_position('top')
axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Transformed Image");axes[1].xaxis.set_ticks_position('top') # Hides the axes
axes[2].plot(X, F, color='blue', label='Transformation Curve')
```

```

x_coords, y_coords = zip(*breakpoints)
axes[2].plot(x_coords, y_coords, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()
plt.tight_layout()
plt.show()

```

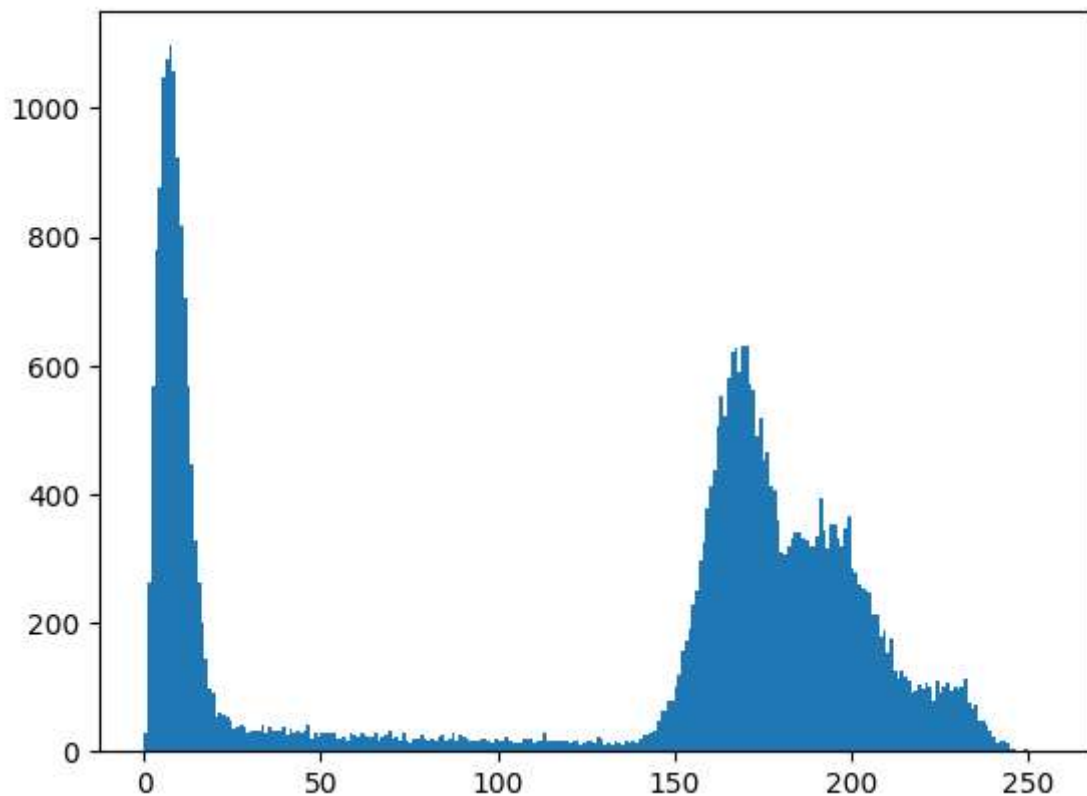


Question 2: Feature Accentuation in Medical Imagery

```

In [118]: brain_img = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)
hist_values, bin_edges, _ = plt.hist(brain_img.ravel(), bins=256, range=[0,256])
plt.show()
for intensity in range(140,256):
    print(f"Intensity {intensity}: {100*hist_values[intensity]/np.max(hist_values)}")

```



Intensity 140: 1.8231540565177757
Intensity 141: 2.3701002734731085
Intensity 142: 2.4612579762989975
Intensity 143: 2.643573381950775
Intensity 144: 2.917046490428441
Intensity 145: 4.375569735642662
Intensity 146: 5.651777575205105
Intensity 147: 5.742935278030994
Intensity 148: 7.201458523245214
Intensity 149: 7.201458523245214
Intensity 150: 9.115770282588878
Intensity 151: 10.847766636280765
Intensity 152: 14.402917046490428
Intensity 153: 15.587967183226983
Intensity 154: 17.502278942570648
Intensity 155: 20.692798541476755
Intensity 156: 22.880583409298087
Intensity 157: 27.07383773928897
Intensity 158: 29.535095715587968
Intensity 159: 34.45761166818596
Intensity 160: 37.55697356426618
Intensity 161: 39.8359161349134
Intensity 162: 46.12579762989973
Intensity 163: 50.22789425706472
Intensity 164: 47.49316317228806
Intensity 165: 52.78030993618961
Intensity 166: 56.60893345487694
Intensity 167: 57.15587967183227
Intensity 168: 53.69188696444849
Intensity 169: 57.429352780309934
Intensity 170: 57.338195077484045
Intensity 171: 52.14220601640839
Intensity 172: 51.13947128532361
Intensity 173: 44.57611668185962
Intensity 174: 47.21969006381039
Intensity 175: 41.20328167730173
Intensity 176: 42.2971741112124
Intensity 177: 37.55697356426618
Intensity 178: 36.91886964448496
Intensity 179: 32.63445761166819
Intensity 180: 28.167730173199637
Intensity 181: 27.80309936189608
Intensity 182: 29.079307201458523
Intensity 183: 30.0820419325433
Intensity 184: 30.993618960802188
Intensity 185: 31.084776663628077
Intensity 186: 30.264357338195076
Intensity 187: 29.808568824065635
Intensity 188: 29.17046490428441
Intensity 189: 29.17046490428441
Intensity 190: 30.355515041020965
Intensity 191: 36.007292616226074
Intensity 192: 31.449407474931633
Intensity 193: 28.805834092980856
Intensity 194: 32.087511394712855
Intensity 195: 32.178669097538744
Intensity 196: 30.173199635369187
Intensity 197: 28.988149498632634
Intensity 198: 31.63172288058341
Intensity 199: 33.181403828623516
Intensity 200: 26.071103008204194
Intensity 201: 25.250683682771193
Intensity 202: 23.701002734731084

Intensity 203: 23.154056517775754
 Intensity 204: 22.69826800364631
 Intensity 205: 22.42479489516864
 Intensity 206: 19.416590701914313
 Intensity 207: 19.416590701914313
 Intensity 208: 16.40838650865998
 Intensity 209: 17.137648131267092
 Intensity 210: 14.038286235186874
 Intensity 211: 15.952597994530537
 Intensity 212: 11.485870556061988
 Intensity 213: 10.300820419325433
 Intensity 214: 11.577028258887877
 Intensity 215: 10.48313582497721
 Intensity 216: 10.118505013673655
 Intensity 217: 8.295350957155879
 Intensity 218: 8.568824065633546
 Intensity 219: 9.389243391066545
 Intensity 220: 8.842297174111213
 Intensity 221: 9.662716499544212
 Intensity 222: 9.206927985414767
 Intensity 223: 7.292616226071103
 Intensity 224: 9.936189608021877
 Intensity 225: 8.386508659981768
 Intensity 226: 9.298085688240656
 Intensity 227: 9.7538742023701
 Intensity 228: 8.659981768459435
 Intensity 229: 9.115770282588878
 Intensity 230: 9.02461257976299
 Intensity 231: 9.206927985414767
 Intensity 232: 10.209662716499544
 Intensity 233: 6.836827711941659
 Intensity 234: 6.01640838650866
 Intensity 235: 6.472196900638104
 Intensity 236: 4.466727438468551
 Intensity 237: 4.284412032816773
 Intensity 238: 3.7374658158614404
 Intensity 239: 2.917046490428441
 Intensity 240: 2.0054694621695535
 Intensity 241: 1.1850501367365542
 Intensity 242: 1.6408386508659982
 Intensity 243: 1.3673655423883317
 Intensity 244: 1.1850501367365542
 Intensity 245: 0.27347310847766637
 Intensity 246: 0.45578851412944393
 Intensity 247: 0.18231540565177756
 Intensity 248: 0.0
 Intensity 249: 0.27347310847766637
 Intensity 250: 0.0
 Intensity 251: 0.09115770282588878
 Intensity 252: 0.0
 Intensity 253: 0.0
 Intensity 254: 0.0
 Intensity 255: 0.0

```

In [119]: f = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)
          print(f)
          breakpoints = [(0, 0), (153, 0), (168, 255), (185, 0), (221, 0)]

          transformed_img, X, F = intensity_transform(f, breakpoints)
          fig, axes = plt.subplots(1, 3, figsize=(15, 5))

          axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
          axes[0].set_title("Original Image")
  
```

```

axes[0].xaxis.set_ticks_position('top')

axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Accentuate White Matter")
axes[1].xaxis.set_ticks_position('top')

axes[2].plot(X, F, color='blue', label='Transformation Curve')
x_coords, y_coords = zip(*breakpoints)
axes[2].plot(x_coords, y_coords, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()

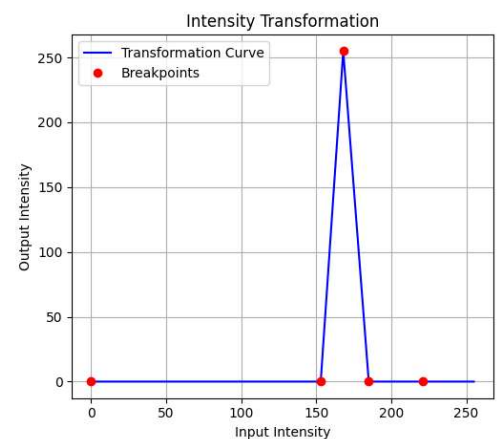
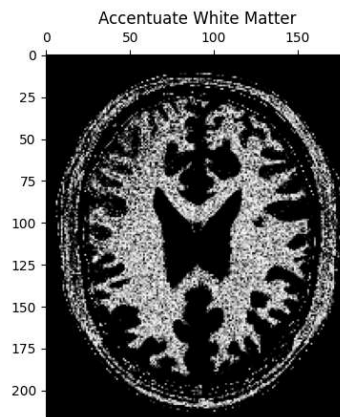
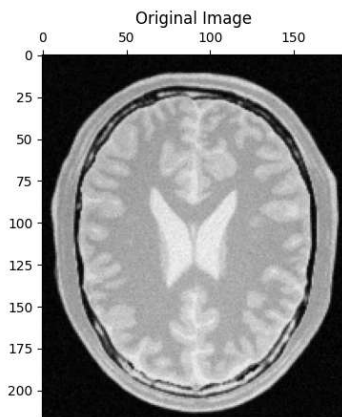
plt.tight_layout()
plt.show()

```

```

[[ 1 10 17 ... 2 13 2]
 [12 10 2 ... 6 8 7]
 [12 8 13 ... 2 6 7]
 ...
 [ 7 11 6 ... 6 5 1]
 [ 9 8 4 ... 2 4 2]
 [12 13 10 ... 2 6 5]]

```



```

In [120]: f = cv.imread("images/brain.png", cv.IMREAD_GRAYSCALE)
print(f)
breakpoints = [(0, 0), (185, 0), (194, 255), (222, 0), (255, 0)]

transformed_img, X, F = intensity_transform(f, breakpoints)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].xaxis.set_ticks_position('top')

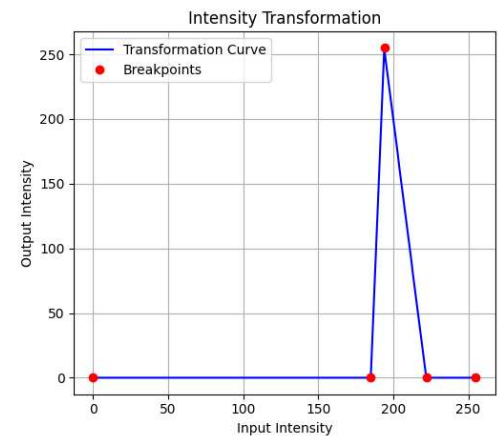
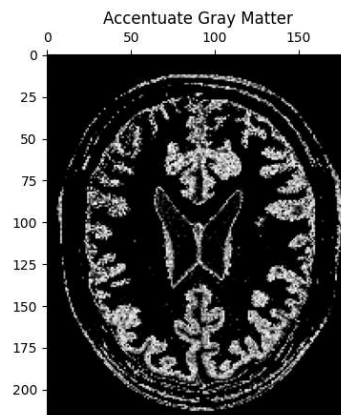
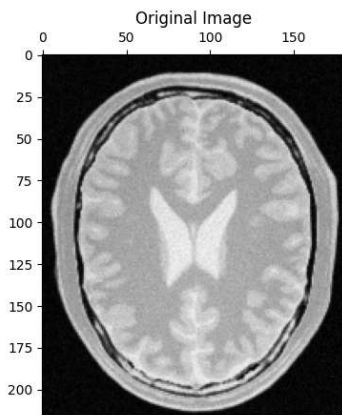
axes[1].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Accentuate Gray Matter")
axes[1].xaxis.set_ticks_position('top')

axes[2].plot(X, F, color='blue', label='Transformation Curve')
x_coords, y_coords = zip(*breakpoints)
axes[2].plot(x_coords, y_coords, 'ro', label='Breakpoints')
axes[2].set_title("Intensity Transformation")
axes[2].set_xlabel("Input Intensity")
axes[2].set_ylabel("Output Intensity")
axes[2].grid(True)
axes[2].legend()

```

```
plt.tight_layout()
plt.show()
```

```
[ [ 1 10 17 ... 2 13 2]
  [12 10 2 ... 6 8 7]
  [12 8 13 ... 2 6 7]
  ...
  [ 7 11 6 ... 6 5 1]
  [ 9 8 4 ... 2 4 2]
  [12 13 10 ... 2 6 5]]
```



Interpretation of Question 2: Tissue Accentuation

To analyze specific tissues in the brain proton density image, I first plotted the histogram to identify the distinct intensity clusters.

- **White Matter:** On the histogram, I identified this as the first major peak. I designed a triangular breakpoint filter ranging from 153 to 185, peaking at an intensity of **168**.
- **Gray Matter:** identified this as the second peak on the histogram and designed a triangular breakpoint filter ranging from 185 to 222, peaking at an intensity of **194**.

So by applying these custom breakpoint coordinates, the target matter is driven to 255 (white) while all other background and tissue values are suppressed to 0 (black).

Question 3: Gamma Correction in Lab Color Space

```
In [121]: img_original = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\white.jpg", cv.IMREAD_COLOR)
f = cv.cvtColor(img_original, cv.COLOR_BGR2LAB)
L,a,b = cv.split(f)
gamma = 0.5
Lc = (((L/255.0)**(gamma))*255).astype(np.uint8)
img_lab = cv.merge([Lc,a,b])
img = cv.cvtColor(img_lab, cv.COLOR_LAB2RGB)

# Create a figure with 2 subplots side-by-side
# Create a figure with a 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(12, 9))

# 1. Top-Left: Original Image
axes[0, 0].imshow(cv.cvtColor(img_original, cv.COLOR_BGR2RGB))
axes[0, 0].set_title("Original Image")
axes[0, 0].axis('off') # Hides the axes and border completely

# 2. Top-Right: Edited Image
```

```

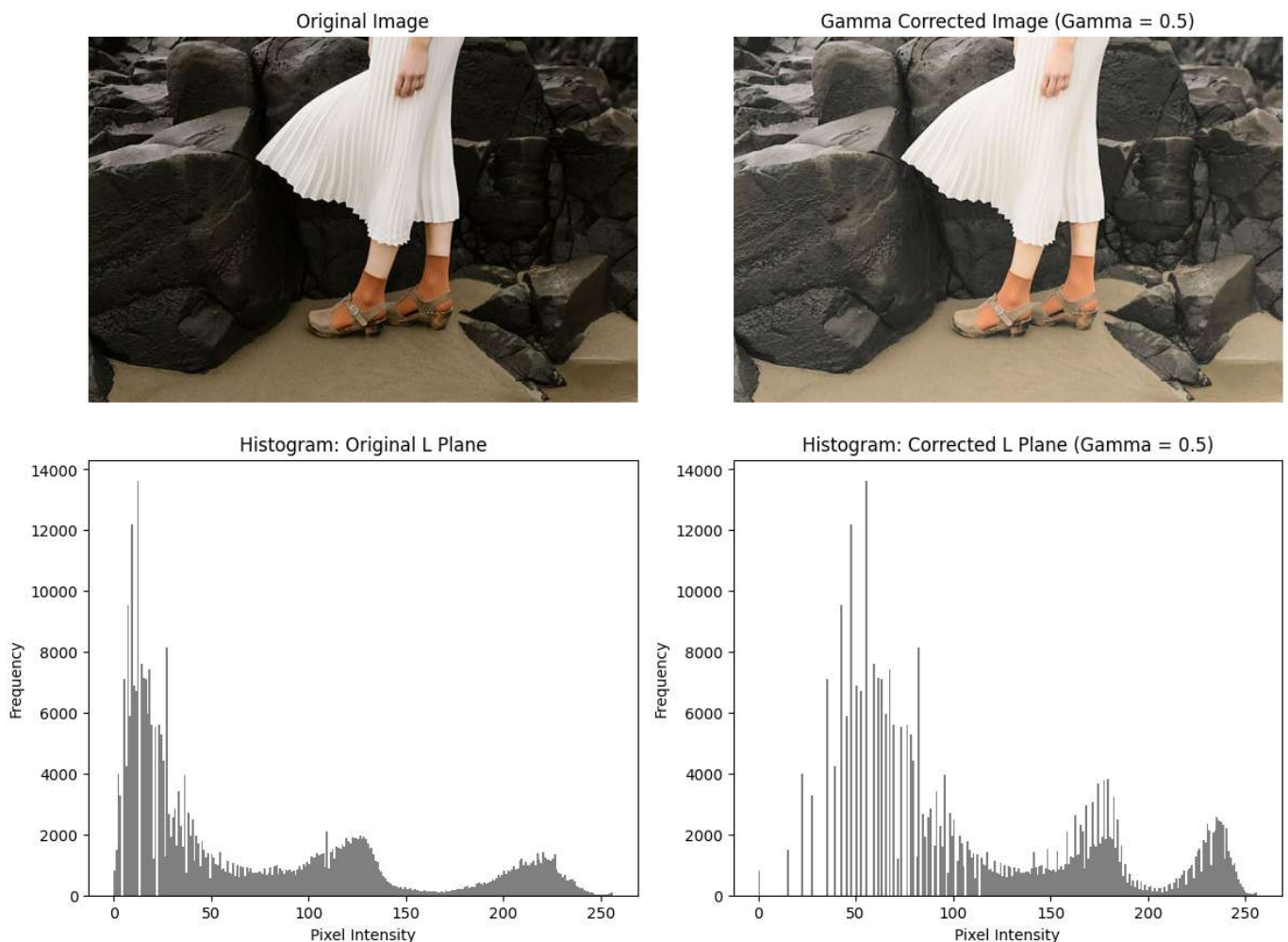
axes[0, 1].imshow(img)
axes[0, 1].set_title(f"Gamma Corrected Image (Gamma = {gamma})")
axes[0, 1].axis('off')

# 3. Bottom-Left: Histogram for the Original L plane
axes[1, 0].hist(L.ravel(), bins=256, range=[0, 256], color='gray')
axes[1, 0].set_title("Histogram: Original L Plane")
axes[1, 0].set_xlabel("Pixel Intensity")
axes[1, 0].set_ylabel("Frequency")

# 4. Bottom-Right: Histogram for the Corrected L plane
axes[1, 1].hist(Lc.ravel(), bins=256, range=[0, 256], color='gray')
axes[1, 1].set_title(f"Histogram: Corrected L Plane (Gamma = {gamma})")
axes[1, 1].set_xlabel("Pixel Intensity")
axes[1, 1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()

```



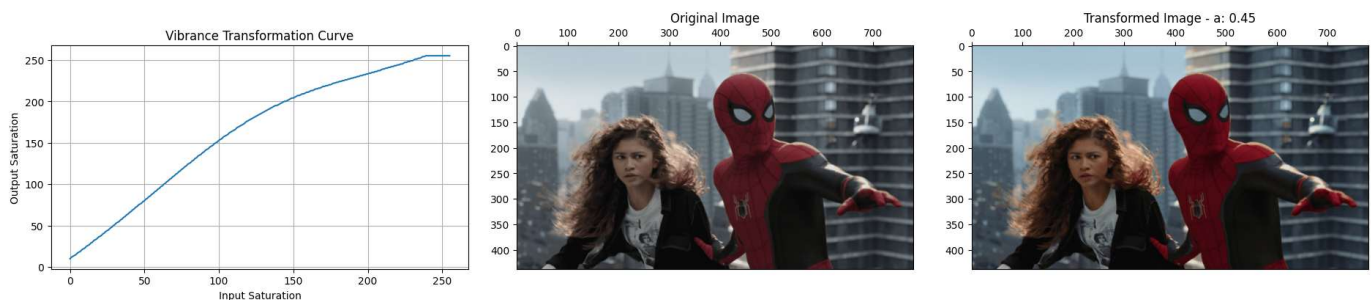
Interpretation of Question 3: Gamma Correction

The goal of this operation was to reveal the shadow details in the underexposed rocks without distorting the colors of the woman's skin or dress. To achieve this, the image was converted to the $L^*a^*b^*$ color space, separating the luminance (L^*) from the chrominance (a^* , b^*).

A gamma correction of $\gamma = 0.5$ was applied strictly to the normalized L plane. Because $\gamma < 1$, this non-linear power-law transformation stretches the lower intensity values. As shown in the histograms, the heavy spike of dark pixels near 0 in the original L plane is spread out towards the mid-tones in the corrected L plane, effectively brightening the shadows while maintaining the original color data.

Question 4: Vibrance Enhancement via Saturation Transformation

```
In [122]: f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\spider.png", cv.IMREAD_COLOR)
img = cv.cvtColor(f, cv.COLOR_BGR2HSV)
h,s, v = cv.split(img)
sigma = 70
a = 0.45
i = np.arange(256)
F = i + a * (128 * np.exp(-((i-128)**2) / (2*sigma**2)))
F = np.clip(F,0,255).astype(np.uint8)
s_new = F[s]
img_new = cv.merge([h,s_new,v])
img_rgb = cv.cvtColor(img_new, cv.COLOR_HSV2RGB)
fig, axes = plt.subplots(1,3, figsize=(19,4))
axes[0].plot(i, F);axes[0].set_title("Vibrance Transformation Curve")
axes[0].set_xlabel("Input Saturation");axes[0].set_ylabel("Output Saturation")
axes[0].grid(True) # Adds a grid for better readability
axes[1].imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB)); axes[1].xaxis.set_ticks_position('top') ;
axes[1].set_title("Original Image")
axes[2].imshow(img_rgb); axes[2].xaxis.set_ticks_position('top')
axes[2].set_title(f"Transformed Image - a: {a}")
plt.tight_layout()
plt.show()
```



To achieve a visually pleasing output that enhances the mid-tone colors of the image without oversaturating the bright reds, I adjusted the parameter to $a = 0.4$

Question 5: Global Histogram Equalization

```
In [123]: def equalize(f):
    hist, _ = np.histogram(f.flatten(),256,[0,256],True)
    cum_sum = np.cumsum(hist)*(255)
    F = cum_sum.astype(np.uint8)
    img = F[f]
    return img

f =cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\shells.tif", cv.IMREAD_GRAYSCALE)
img_eq = equalize(f)
fig, axes = plt.subplots(1, 4, figsize=(20, 5))

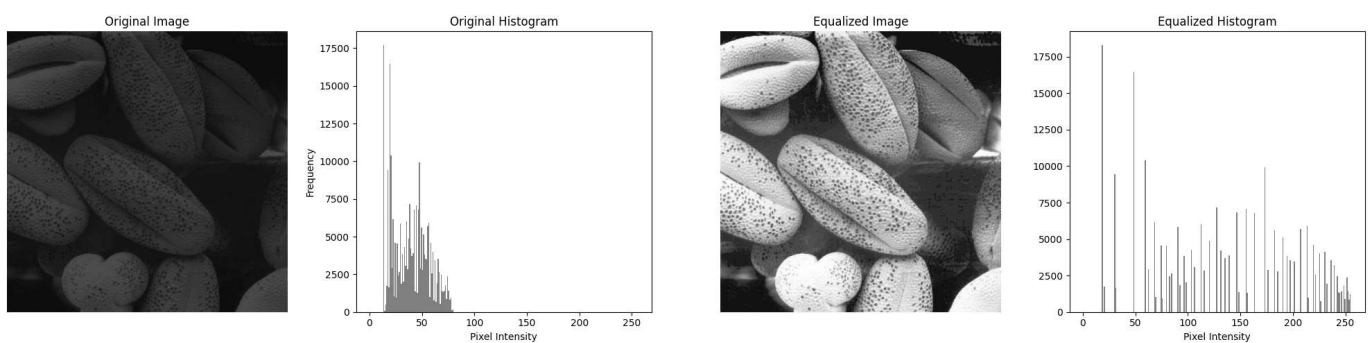
# 1. Original Image
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].axis("off")
```

```
# 2. Original Histogram
axes[1].hist(f.flatten(), bins=256, range=[0, 256], color='gray')
axes[1].set_title("Original Histogram")
axes[1].set_xlabel("Pixel Intensity")
axes[1].set_ylabel("Frequency")

# 3. Equalized Image
axes[2].imshow(img_eq, cmap='gray', vmin=0, vmax=255)
axes[2].set_title("Equalized Image")
axes[2].axis("off")

# 4. Equalized Histogram
axes[3].hist(img_eq.flatten(), bins=256, range=[0, 256], color='gray')
axes[3].set_title("Equalized Histogram")
axes[3].set_xlabel("Pixel Intensity")

plt.tight_layout()
plt.show()
```



Question 6: Foreground-Aware Histogram Equalization

```
In [124]: f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\jeniffer.jpg", cv.IMREAD_COLOR)
h, s, v = cv.split(cv.cvtColor(f, cv.COLOR_BGR2HSV))
fig, axes = plt.subplots(1,3, figsize=(15,6))
axes[0].imshow(h, cmap='gray', vmin=0, vmax=255); axes[0].xaxis.set_ticks_position('top'); axes
axes[1].imshow(s, cmap='gray', vmin=0, vmax=255); axes[1].axis(False); axes[1].set_title("Saturation")
axes[2].imshow(v, cmap='gray', vmin=0, vmax=255); axes[2].axis(False); axes[2].set_title("Value")
plt.tight_layout()
plt.show()
```



By observing above three graphs, it can be concluded that the Saturation plane is optimal to mask the foreground.

```
In [125]: _, mask = cv.threshold(s, 11, 255, cv.THRESH_BINARY)
foreg_pixels = v[mask == 255]
hist, _ = np.histogram(foreg_pixels, 256, [0, 256], True)
cum_sum = np.cumsum(hist)*(255)
F = cum_sum.astype(np.uint8)
```

```

v_eq = v.copy()
v_eq[mask==255] = F[v[mask==255]]

img_rgb_eq = cv.cvtColor(cv.merge([h,s,v_eq]), cv.COLOR_HSV2RGB)

print(F.shape)
fig, axes = plt.subplots(1, 5, figsize=(22, 4))

# Plot 1: Original V Channel Histogram
axes[0].hist(v.flatten(), bins=256, color='gray')
axes[0].set_title("Original V Histogram")

# Plot 2: Foreground-Only Histogram
axes[1].hist(foreg_pixels, bins=256, color='blue')
axes[1].set_title("Foreground Pixels Only")

# Plot 3: Original Image (Fixed the BGR issue for Matplotlib)
axes[2].imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
axes[2].set_title("Original Image")
axes[2].axis("off")

# Plot 4: Final Equalized Image
axes[3].imshow(img_rgb_eq)
axes[3].set_title("Foreground Equalized Image")
axes[3].axis("off")

axes[4].hist(v_eq.flatten(), bins=256, color='gray')

plt.tight_layout()
plt.show()

```

(256,)



Question 7: Edge Detection via Sobel Operator and Kernel Separability

```

In [126]: f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\ein.png", cv.IMREAD_GRAYSCALE)
print(f.shape)
sobel_kernel = np.array([[ 1, 0, -1],
                          [2, 0, -2],
                          [ 1, 0, -1]])
img_filtered = cv.filter2D(f, cv.CV_64F, sobel_kernel)
img_filtered_abs = np.abs(img_filtered)
img_filtered_final = np.clip(img_filtered_abs, 0, 255).astype(np.uint8)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].axis("off")

axes[1].imshow(img_filtered_final, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Sobel Filtered (Vertical Edges)")

```

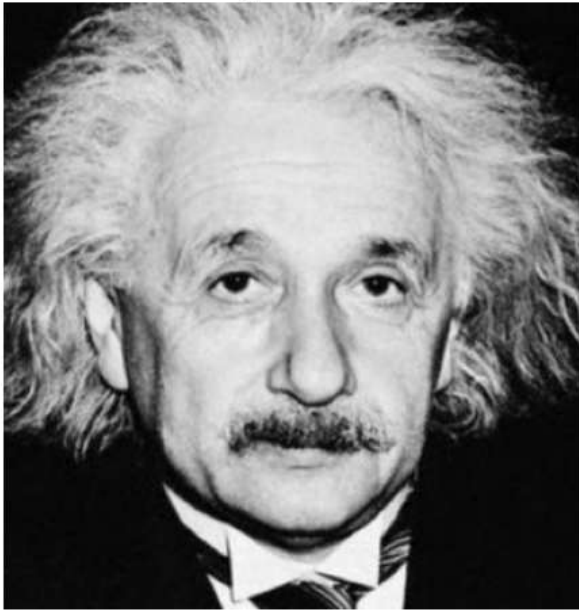
```
axes[1].axis("off")
```

```
plt.tight_layout()
```

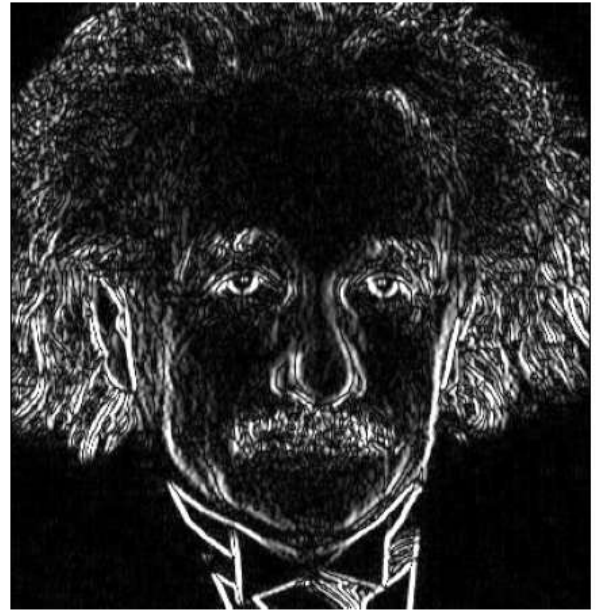
```
plt.show()
```

(380, 364)

Original Image



Sobel Filtered (Vertical Edges)



```
In [127]: def sobel(img, kernel):
            rows, cols = img.shape
            output = np.zeros((rows, cols), dtype=np.float64)
            pad_img = np.pad(img, pad_width=1, mode='constant', constant_values=0)
            for i in range(rows):
                for j in range(cols):
                    region = pad_img[i:i+3, j:j+3]
                    output[i, j] = np.sum(region * kernel)
            return output

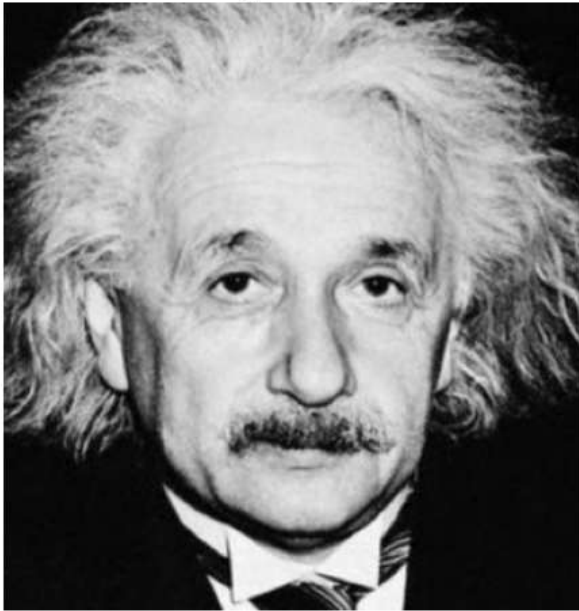
f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\ein.png", cv.IMREAD_GRAYSCALE)
sobel_kernel = np.array([[ 1, 0, -1],
                          [ 2, 0, -2],
                          [ 1, 0, -1]])

sobel_result = sobel(f, sobel_kernel)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(f, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original Image")
axes[0].axis("off")

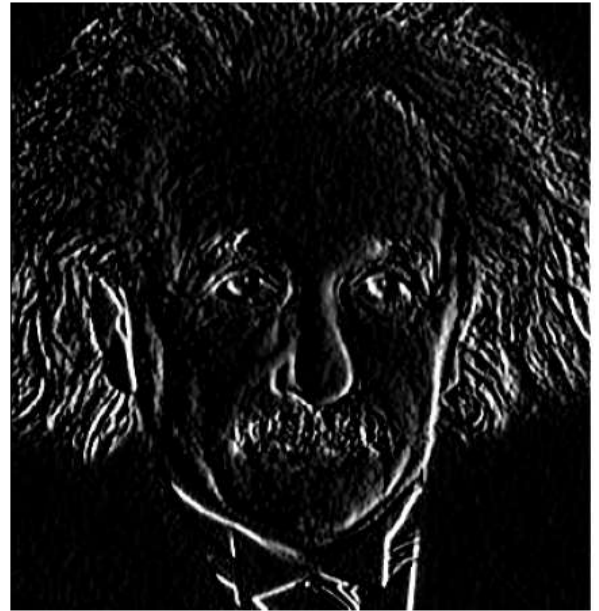
axes[1].imshow(sobel_result, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("Sobel Filtered (Vertical Edges)")
axes[1].axis("off")

plt.tight_layout()
plt.show()
```

Original Image

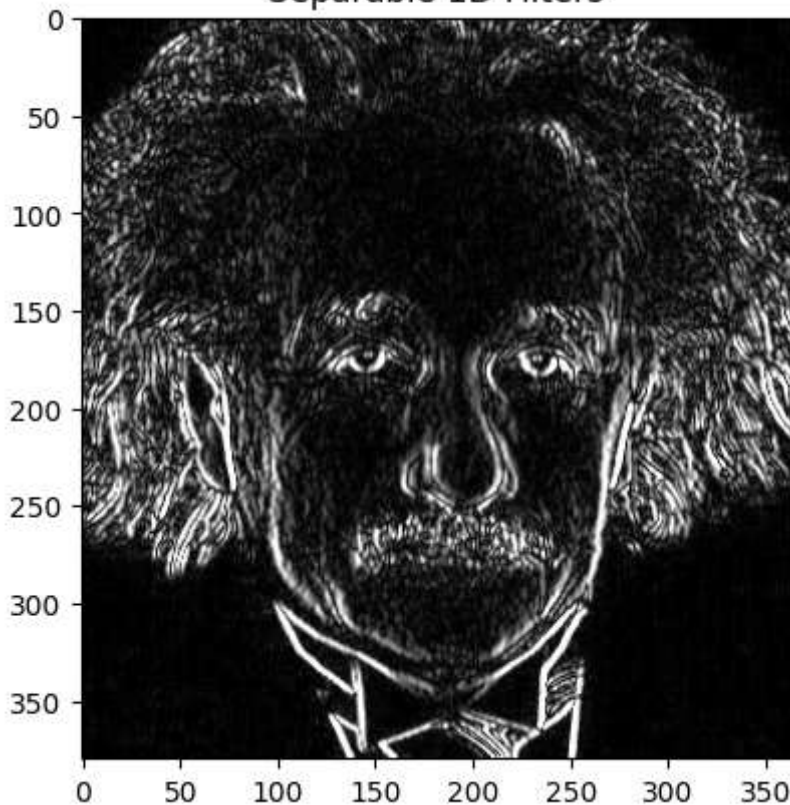


Sobel Filtered (Vertical Edges)



```
In [128]: v_kernel = np.array([[1],
                                [2],
                                [1]])
h_kernel = np.array([[1, 0, -1]])
conv1 = cv.filter2D(f, cv.CV_64F, h_kernel)
img_filtered = cv.filter2D(conv1, cv.CV_64F, v_kernel)
img_abs = np.abs(img_filtered)
img = np.clip(img_abs, 0, 255).astype(np.uint8)
plt.imshow(img, cmap='gray', vmin=0, vmax=255)
plt.title("Separable 1D Filters")
plt.show()
```

Separable 1D Filters



Question 8: Image Scaling and Interpolation Evaluation

```
In [129]: def zoom_image(img, scale_factor, method='nearest'):
    new_width = int(img.shape[1] * scale_factor)
    new_height = int(img.shape[0] * scale_factor)

    if method == 'nearest':
        interp = cv.INTER_NEAREST
    elif method == 'bilinear':
        interp = cv.INTER_LINEAR
    zoomed_img = cv.resize(img, (new_width, new_height), interpolation=interp)
    return zoomed_img

def compute_normalized_ssd(img_original, img_zoomed):
    orig_f = img_original.astype(np.float64)
    zoom_f = img_zoomed.astype(np.float64)
    ssd = np.sum((orig_f - zoom_f)**2)
    norm_factor = np.sum(orig_f**2)
    return ssd / norm_factor
```

```
In [130]: image_pairs = [
    (r"D:\Semester 5\CV\EN3160_Assignment_01\images\im01small.png",
     r"D:\Semester 5\CV\EN3160_Assignment_01\images\im01.png"),

    (r"D:\Semester 5\CV\EN3160_Assignment_01\images\im02small.png",
     r"D:\Semester 5\CV\EN3160_Assignment_01\images\im02.png")
]

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

scale_factor = 4

for i, (path_small, path_large) in enumerate(image_pairs):
    # Load and convert images to RGB
    img_small = cv.cvtColor(cv.imread(path_small, cv.IMREAD_COLOR), cv.COLOR_BGR2RGB)
    img_large = cv.cvtColor(cv.imread(path_large, cv.IMREAD_COLOR), cv.COLOR_BGR2RGB)

    img_nn = zoom_image(img_small, scale_factor, method='nearest')
    img_bl = zoom_image(img_small, scale_factor, method='bilinear')

    ssd_nn = compute_normalized_ssd(img_large, img_nn)
    ssd_bl = compute_normalized_ssd(img_large, img_bl)

    axes[i, 0].imshow(img_large)
    axes[i, 0].set_title(f"Pair {i+1}: Original High-Res")
    axes[i, 0].axis("off")

    axes[i, 1].imshow(img_nn)
    axes[i, 1].set_title(f"Nearest-Neighbor (SSD: {ssd_nn:.4f})")
    axes[i, 1].axis("off")

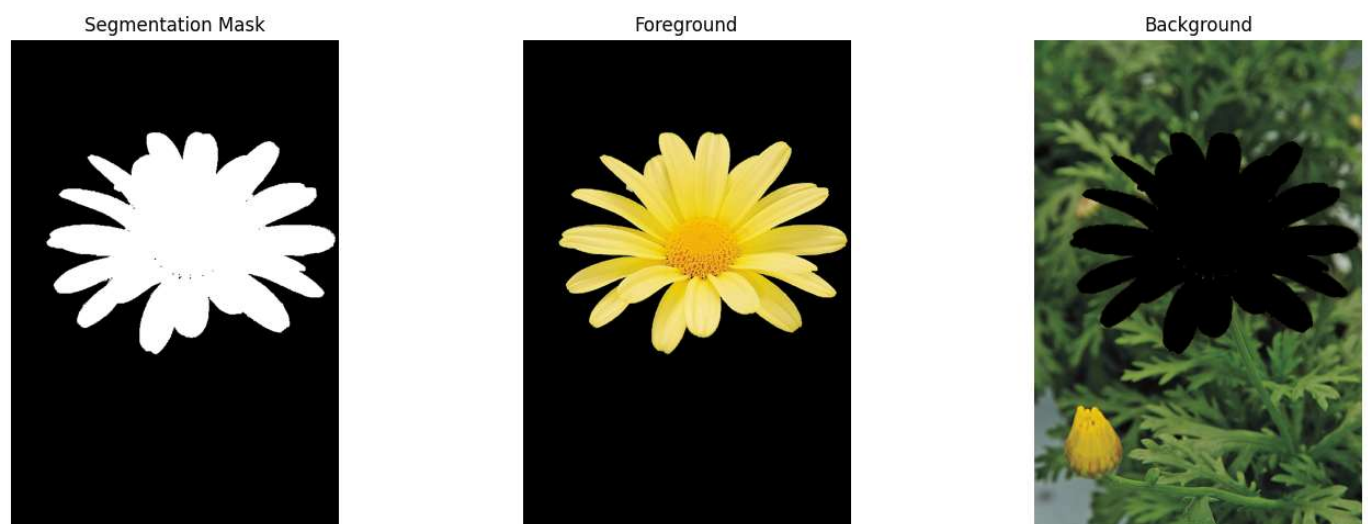
    axes[i, 2].imshow(img_bl)
    axes[i, 2].set_title(f"Bilinear (SSD: {ssd_bl:.4f})")
    axes[i, 2].axis("off")

plt.tight_layout()
plt.show()
```



Question 9: GrabCut Segmentation and Depth-of-Field Enhancement

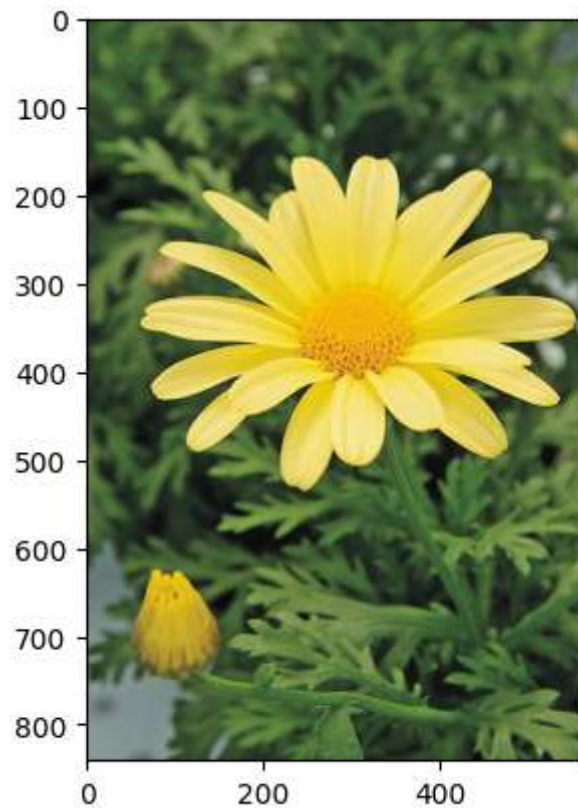
```
In [131]: img_flower = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\daisy.jpg")
img_rgb = cv.cvtColor(img_flower, cv.COLOR_BGR2RGB)
mask = np.zeros(img_flower.shape[:2], np.uint8)
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)
rect = (20, 130, 535, 470)
cv.grabCut(img_flower, mask, rect, bgdModel, fgdModel, 5, cv.GC_INIT_WITH_RECT)
mask_binary = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')
foreground = cv.bitwise_and(img_rgb, img_rgb, mask=mask_binary)
mask_inv = cv.bitwise_not(mask_binary * 255) # Invert mask for background
background = cv.bitwise_and(img_rgb, img_rgb, mask=mask_inv)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(mask_binary, cmap='gray'); axes[0].set_title("Segmentation Mask"); axes[0].axis('off')
axes[1].imshow(foreground); axes[1].set_title("Foreground"); axes[1].axis('off')
axes[2].imshow(background); axes[2].set_title("Background"); axes[2].axis('off')
plt.tight_layout()
plt.show()
```



```
In [132]: f = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\daisy.jpg")
plt.imshow(cv.cvtColor(f, cv.COLOR_BGR2RGB))
```

```
print(f.shape)
```

```
(841, 561, 3)
```



```
In [133]: # 1. Apply a heavy Gaussian blur to the ORIGINAL image
          blurred_img = cv.GaussianBlur(img_rgb, (51, 51), 0)

          # 2. Combine the sharp foreground with the blurred background
          # We expand the 2D mask to 3D so it matches the color image channels
          mask_3d = mask_binary[:, :, np.newaxis]

          # Where mask is 1, use original pixel. Where mask is 0, use blurred pixel.
          enhanced_img = np.where(mask_3d == 1, img_rgb, blurred_img)

          # --- Plotting Part (b) ---
          fig, axes = plt.subplots(1, 2, figsize=(12, 6))
          axes[0].imshow(img_rgb); axes[0].set_title("Original Image"); axes[0].axis('off')
          axes[1].imshow(enhanced_img); axes[1].set_title("Enhanced (Portrait Mode)"); axes[1].axis('off')
          plt.tight_layout()
          plt.show()
```

Original Image



Enhanced (Portrait Mode)



9(c) Discussion: The "Dark Halo" Artifact

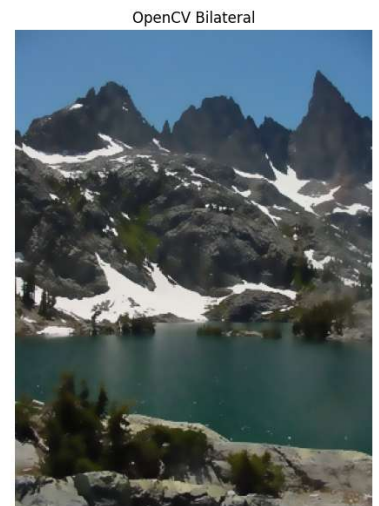
The dark region just beyond the edge of the flower in the enhanced image is an artifact caused by the spatial filtering process, often referred to as "color bleeding" or a "halo effect."

When the heavy Gaussian blur is applied to the original image to create the artificial depth-of-field, the filter calculates a weighted average of neighboring pixels. Along the boundary of the flower, the dark pixels from the flower's edges and internal shadows are averaged together with the adjacent lighter background pixels. This spatial mixing artificially darkens the background pixels immediately surrounding the flower.

When the sharp, segmented foreground mask is superimposed back onto this blurred image, the darkened background pixels just outside the segmentation boundary remain visible, resulting in the dark halo effect.

Question 10: Edge-Preserving Smoothing via Bilateral Filtering

```
In [134]: img_lake = cv.imread(r"D:\Semester 5\CV\EN3160_Assignment_01\images\minaret.jpg", cv.IMREAD_COLOR)
img_rgb = cv.cvtColor(img_lake, cv.COLOR_BGR2RGB)
img_gaussian = cv.GaussianBlur(img_rgb, (9, 9), sigmaX=1)
img_bilateral_cv = cv.bilateralFilter(img_rgb, d=9, sigmaColor=75, sigmaSpace=15)
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
axes[0].imshow(img_rgb); axes[0].set_title("Original Image"); axes[0].axis('off')
axes[1].imshow(img_gaussian); axes[1].set_title("Gaussian Blur"); axes[1].axis('off')
axes[2].imshow(img_bilateral_cv); axes[2].set_title("OpenCV Bilateral"); axes[2].axis('off')
plt.tight_layout()
plt.show()
```



```
In [135]: def custom_bilateral_filter_channel(img, d, sigma_color, sigma_space):
    """Applies bilateral filter to a single 2D channel."""
    img = img.astype(np.float64)
    rows, cols = img.shape
    output = np.zeros((rows, cols), dtype=np.float64)
    r = d // 2
    x, y = np.meshgrid(np.arange(-r, r+1), np.arange(-r, r+1))
    spatial_weights = np.exp(-(x**2 + y**2) / (2 * sigma_space**2))
    padded_img = np.pad(img, pad_width=r, mode='reflect')
    for i in range(rows):
        for j in range(cols):
            patch = padded_img[i:i+d, j:j+d]
            center_val = padded_img[i+r, j+r]
            range_weights = np.exp(-((patch - center_val)**2) / (2 * sigma_color**2))
            total_weights = spatial_weights * range_weights
            output[i, j] = (np.sum(total_weights * patch) / np.sum(total_weights)).astype(np.float64)

    return output

def custom_bilateral_color(img_rgb, d, sigma_color, sigma_space):
    """Splits color image, filters channels, and merges."""
    r, g, b = cv.split(img_rgb)
    r_filtered = custom_bilateral_filter_channel(r, d, sigma_color, sigma_space)
    g_filtered = custom_bilateral_filter_channel(g, d, sigma_color, sigma_space)
    b_filtered = custom_bilateral_filter_channel(b, d, sigma_color, sigma_space)

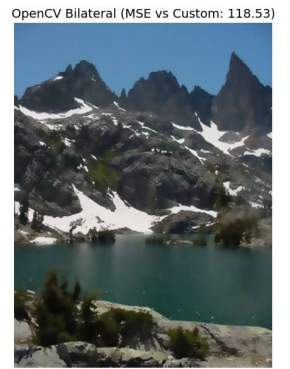
    merged = cv.merge([r_filtered, g_filtered, b_filtered])
    return np.clip(merged, 0, 255).astype(np.uint8)
```

```
In [136]: img_bilateral_custom = custom_bilateral_color(img_rgb, d=9, sigma_color=75, sigma_space=15)
mse = np.mean((img_bilateral_cv.astype("float") - img_bilateral_custom.astype("float")) ** 2)
print(f"Mean Squared Error (OpenCV vs Custom): {mse:.4f}")

fig, axes = plt.subplots(1, 4, figsize=(24, 6))
axes[0].imshow(img_rgb)
axes[0].set_title("Original Image", fontsize=14)
axes[0].axis('off')
axes[1].imshow(img_gaussian)
axes[1].set_title("Gaussian Blur", fontsize=14)
axes[1].axis('off')
axes[2].imshow(img_bilateral_custom)
axes[2].set_title("Custom Bilateral Filter", fontsize=14)
axes[2].axis('off')
axes[3].imshow(img_bilateral_cv)
axes[3].set_title(f"OpenCV Bilateral (MSE vs Custom: {mse:.2f})", fontsize=14)
axes[3].axis('off')
```

```
plt.tight_layout()
plt.show()
```

Mean Squared Error (OpenCV vs Custom): 118.5314



10(b) Discussion: Custom vs. OpenCV Bilateral Filter

Custom bilateral filter has smoothed while preserving the edges to some extent. It has occurred a painterly effect. The mean squared error is 118.5 .

This larger MSE created due to the way that the R,G,B color channels are handled. In the custom method, it processes these channels separately and calculat the range weights. This creates minor color bleeding variations near edges. But OpenCV's bilateral filter calculutes the internsity difference using L_2 norm of the 3 channel color vectors and apply unified weuight which perfectly preserves the original hue.